HW9: Individual Effort Report

***Server Design***

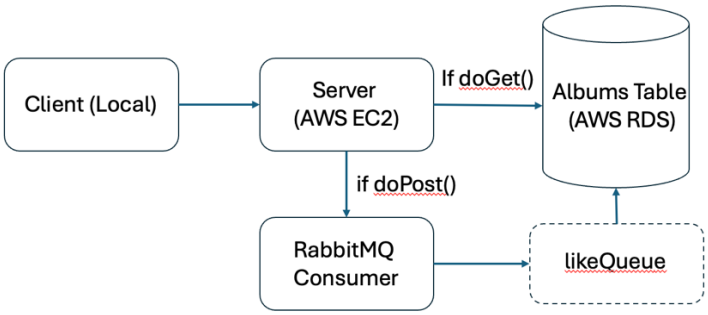
I. Overview of Major Components and Packages

1. Server.java (Servlet)
    - Handles HTTP requests (GET/POST).
    - Interacts with the MySQL database.
    - Publishes review events to RabbitMQ for asynchronous processing.
2. AlbumInfo.java
    - Model class representing album metadata.
    - Used as a data transfer object (DTO) for serializing/deserializing album data (via Gson).
3. ReviewConsumer.java
    - Consumes messages from RabbitMQ (likeQueue).
    - Parses and stores user reviews into the MySQL album_reviews table.

II. Data Flow and Relationships

1. Album Upload
    - Stores the album into the albums table in MySQL.
    - Returns AlbumInfo as a JSON response.
2. Review Upload
    - Publishes it to RabbitMQ queue likeQueue.
3. Review Consumption (RabbitMQ Consumer)
    - On message receive, inserts or updates the review in the MySQL album_reviews table.
4. Review Get
    - Get random album review stats in JSON

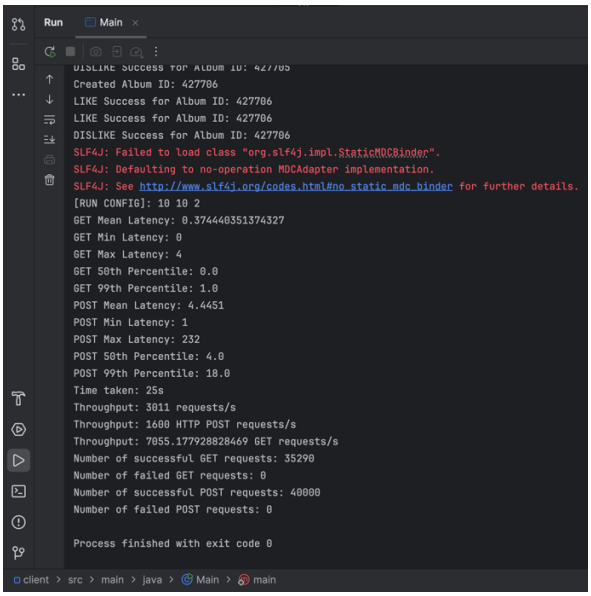
III. Server on AWS

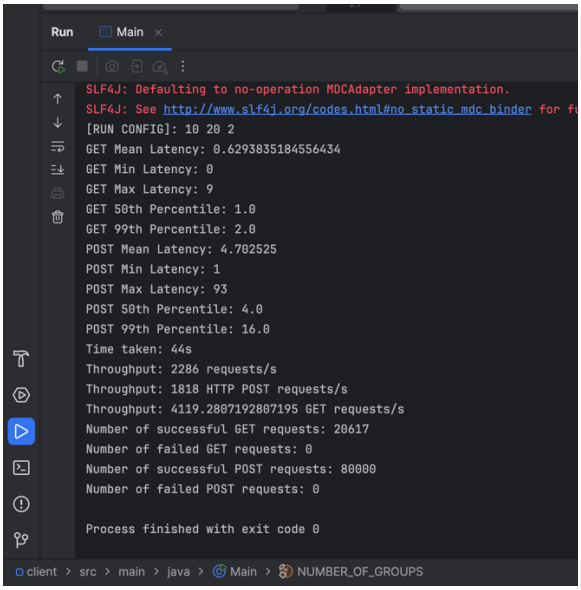


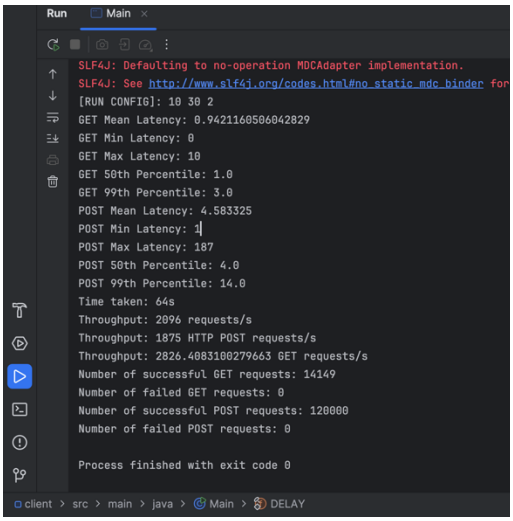

(Note: RabbitMQ runs locally via an SSH channel directing messages)

***Results (locally)***

The throughput has been steadily kept at around 2000 in each experiment, except it jumped to around 3000 in 10 10 2, meaning that the system may have some room to grow, it could be due to queued tasks can be fed more consistently or more parallelism fills in idle CPU/network gaps, or simple the network behaves better at that time. The sure thing right now is that the system may still have room for handling more threads.



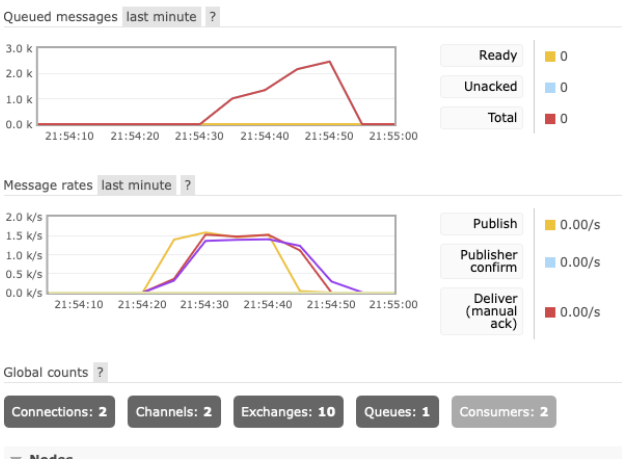





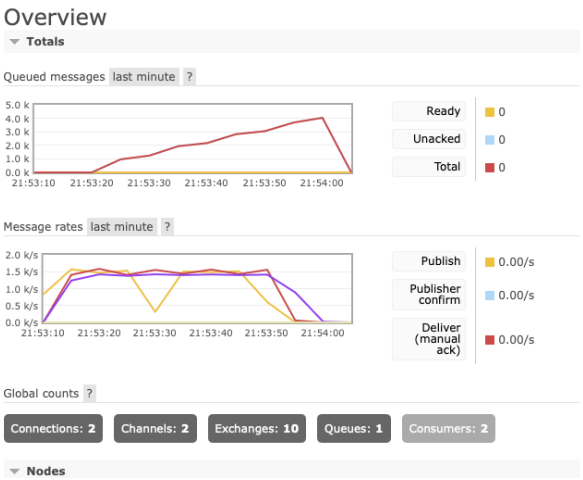

However, RabbitMQ queue sizes increased significantly. Before implementing the 3 getReview threads, the max size was 75. Right now the queue size easily reached 2000, and max at around 7500 in 10302.

Possible reasons maybe that before adding the threads, we had only POST threads creating albums and immediately queuing album IDs so that each post thread finished fast and the album ID queue (albumIds) grew slowly and stopped growing once POST threads finished. After adding the threads, all albums created during POST are already queued by then. And the get review threads Continuously read album IDs *without removing them* from the queue and do not consume(resume) IDs, and allow the queue to grow indefinitely.

RabbitMQ statistics for 10 10 2:



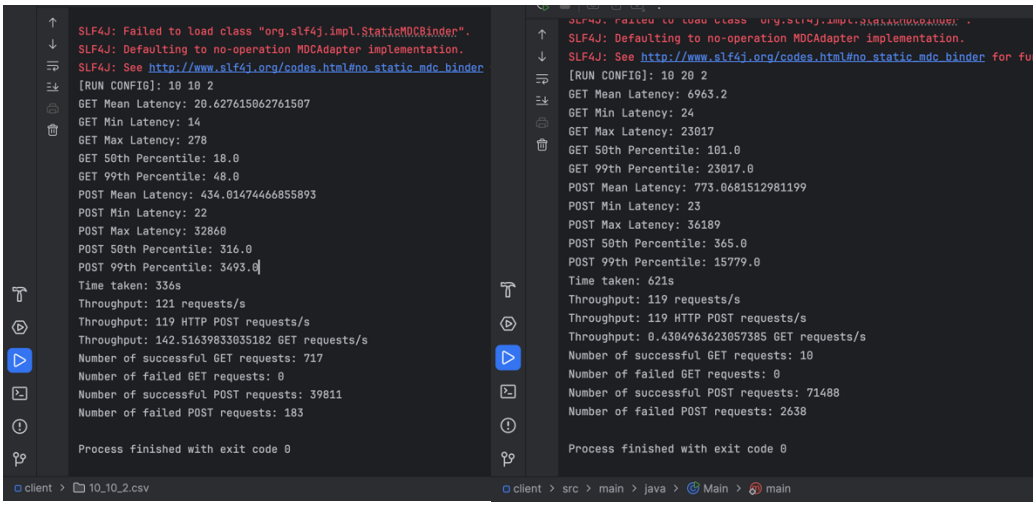

RabbitMQ statistics for 10 20 2:




RabbitMQ statistics for 10 30 2:

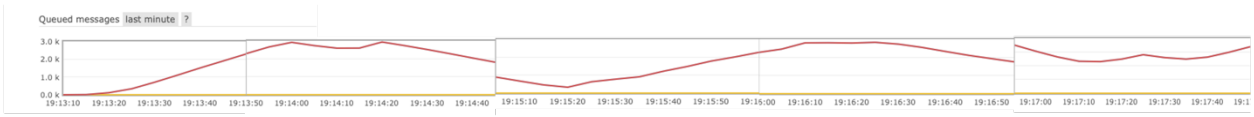

### *Results (AWS EC2)*

The throughput has been steadily kept at around 120 in each experiment, except the connection between local client and server on AWS EC2 stopped during running 10302, resulting in unknown throughput. The sure thing is that the throughput of 10302 is highly unlikely to be better due to queue size. The reason of the dramatically declined throughput may due to the capacity of the AWS EC2 instance, which is of type t2.micro, often good for low-load, burstable tasks.

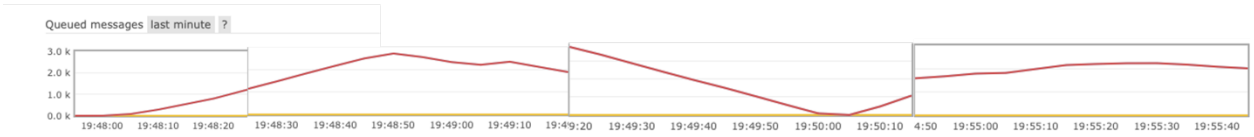



Because we had a lower performance machine, we can see that the queue size also increased. It easily reached around 5000, and the min process rate can reach 1.6 requests/s while the total number of successful get requests is 10. There could be a couple of reasons: 1. There is additional overhead on SSH tunneling because I am running RabbitMQ locally and the server need to direct message via an SSH tunnel. The SSH tunnels add encryption, and result in higher latency.  2. The RabbitMQ is single-threaded. This could be the biggest contributor to the result as the queue size did not vary as significantly as the throughput comparing with performance on running the program locally and on VM.

RabbitMQ statistics for 10 10 2:




RabbitMQ statistics for 10 20 2:




RabbitMQ statistics for 10 30 2: